

A Unified Physics-Informed Neural Network for Adsorption Kinetics and Isotherm Modeling of Catkin Biofilm

Internal Technical Report · PINN3D Model · September 2026

Abstract

Adsorption of a pollutant onto a biosorbent is conventionally described by two disjoint empirical models: a *kinetic* model that predicts how much is adsorbed as a function of time, and an *isotherm* model that predicts the equilibrium adsorption capacity as a function of initial concentration and dose. These are normally fit as two separate curves from two separate experiments, with no guarantee that their parameters agree with each other physically. This report documents a single neural network — a Physics-Informed Neural Network (PINN) — that takes time, initial concentration, and adsorbent dosage as inputs and predicts adsorbed quantity as output, while being constrained during training to simultaneously obey a pseudo-second-order (PSO) kinetic rate law, a zero initial condition, and a Freundlich equilibrium isotherm. All three physical laws share one set of learnable physical constants (a rate constant k_2 and Freundlich constants k_f , n), so the model is forced to find kinetics and equilibrium behavior that are mutually consistent. We describe the domain science, the architecture and loss design, a full account of the numerical issues encountered and fixed during development (an unstable double-backward pass on Apple's MPS GPU backend, unit-scaling bugs that silently corrupted three of the four loss terms, a degenerate solution that let the rate constant collapse to zero, and an unbounded isotherm exponent that made the model diverge), and the final trained result: $k_2 \approx 0.00636$, $k_f \approx 0.327$, $n \approx 0.486$, with all four loss terms converged to the 10^{-5} – 10^{-7} range.

1. Domain Background

1.1 Adsorption in biosorbent water treatment

Adsorption is the process by which a dissolved pollutant (the *adsorbate* — here, whatever contaminant the catkin biofilm was tested against) accumulates on the surface of a solid material (the *adsorbent*, the catkin biofilm). It is one of the most widely used low-cost water treatment mechanisms because natural or waste-derived biosorbents are cheap, and because the process itself is described by a small number of well-established empirical laws. A complete characterization of an adsorbent requires two things: how fast it adsorbs (kinetics) and how much it can adsorb at saturation (equilibrium/isotherm behavior).

1.2 Adsorption kinetics: the pseudo-second-order (PSO) model

Two kinetic models dominate the adsorption literature: the pseudo-first-order (PFO) model of Lagergren (1898), and the pseudo-second-order (PSO) model popularized by Ho & McKay (1999). This project uses PSO, which states that the rate of adsorption is proportional to the *square* of the remaining unfilled capacity:

$$dq_t/dt = k_2 (q_e - q_t)^2$$

where q_t is the amount adsorbed at time t , q_e is the amount adsorbed at equilibrium, and k_2 is the rate constant. PSO kinetics generally fit biosorption data better than PFO because they implicitly capture chemisorption-like, capacity-limited uptake. The key practical difficulty is that q_e is not a free parameter you can pick — it must be the *same* equilibrium value that the isotherm model predicts for that particular initial concentration and dosage. Classical curve-fitting workflows fit PSO and the isotherm separately and never enforce this.

1.3 Adsorption isotherms: the Freundlich model

An isotherm describes adsorption capacity at equilibrium as a function of the equilibrium concentration remaining in solution. The Freundlich isotherm (Freundlich, 1906) is an empirical power law suited to heterogeneous adsorbent surfaces (as opposed to the Langmuir model, which assumes a single, uniform monolayer):

$$q_e = k_f C_e^n$$

where C_e is the equilibrium concentration left in solution, k_f is the Freundlich capacity constant, and n is an intensity/heterogeneity exponent. A mass balance ties C_e back to the initial concentration C_0 , the amount already adsorbed q_e , and the adsorbent dose V_m : $C_e = C_0 - q_e/V_m$. Because q_e appears on both sides, computing it from k_f and n requires solving a fixed point, which this project does with a damped iterative solve (see Section 4.3).

An important, easily-missed physical constraint: for typical favorable adsorption on a heterogeneous surface, the Freundlich exponent n should be in the sub-linear range ($0 < n < 1$), reflecting diminishing marginal capacity as concentration rises. This project's dataset independently confirms this: with n unconstrained, the model drove it above 2 during training and equilibrium capacity predictions exploded to 40–80 mg/g, roughly 20–30× larger than anything in the real data (max observed adsorption ≈ 2.49 mg/g). This is discussed further in Section 6.

1.4 Why unify kinetics and equilibrium in one model?

Fitting PSO and Freundlich separately, from separate experiments, produces two independent parameter sets with no guarantee of physical consistency — the kinetic model's implied

equilibrium capacity for a given concentration/dose may not match what the isotherm model predicts for the same conditions. A single model that is trained against *both* datasets at once, and whose kinetic equilibrium target is computed from the *same* Freundlich parameters used for the isotherm fit, removes this inconsistency by construction. This is the central idea behind this project's architecture.

2. Machine Learning Background: Physics-Informed Neural Networks

2.1 The problem with plain regression here

The combined dataset has only 14 experimental points (9 kinetics measurements at a fixed concentration/dose, plus 5 isotherm measurements at a fixed time/dose). A standard neural network fit to 14 points, with 3 free inputs, is essentially unconstrained outside the tiny neighborhood of those points — it can memorize the data and do almost anything elsewhere (e.g. predict decreasing adsorption over time, or negative adsorption), because nothing in an ordinary mean-squared-error loss tells it that physics forbids that.

2.2 The PINN idea

A Physics-Informed Neural Network (Raissi, Perdikaris & Karniadakis, 2019) augments the ordinary data-fitting loss with extra loss terms computed not from labeled data, but from a governing differential or algebraic equation the output is supposed to obey. Because the network is a differentiable function of its inputs, automatic differentiation (`torch.autograd.grad`) can compute the network's own partial derivatives (e.g. $\partial q / \partial t$) with no finite-difference approximation, and those derivatives can be plugged directly into the governing equation to form a residual. Minimizing that residual at a large number of unlabeled "collocation" points forces the network to behave like a solution to the physics almost everywhere, not just at the 14 points where data exists. This is precisely what makes a 14-point dataset usable here at all.

2.3 What this project's PINN enforces

The trained network must simultaneously minimize four loss terms:

1. **Data loss** — ordinary MSE against the 14 real measurements.
2. **PSO residual loss** — at 2,500 unlabeled collocation points spanning time $\times$ concentration $\times$ dosage, the residual $dq/dt - k_2(q_e - q_t)^2$ should be zero.
3. **Initial-condition loss** — adsorption must be exactly zero at $t = 0$, for every concentration/dosage combination in the collocation grid.

4. **Isotherm (equilibrium) loss** — at a very long time ($t = 400$, well past the last real measurement at $t = 170$), the network's output should equal the Freundlich equilibrium value q_e computed from the *same* k_f , n used in the PSO term.

3. Data

Two experimental sheets were combined into one 14-row table with four columns (`Time`, `Concentration`, `Dosage`, `Adsorption`): 9 kinetics points (fixed concentration = 40, fixed dosage = 1/24) spanning $t = 0\text{--}170$ min, and 5 isotherm points (fixed time = 170 min, fixed dosage = 1/10) spanning concentration = 20–60. All inputs are standardized (zero mean, unit variance) before being fed to the network; the output is standardized the same way, and un-standardized wherever it needs to be compared against a physically-real quantity (see Section 5).

	Time (min)	Concentration	Dosage	Adsorption (mg/g)
count	14	14	14	14
mean	104.3	40.0	0.0625	1.124
std	69.7	8.77	0.029	0.602
min	0	20	0.0417	0.000
max	170	60	0.100	2.485

2,500 unlabeled collocation points (a $25 \times 25 \times 4$ grid over $\text{time} \in [0, 200]$, $\text{concentration} \in [15, 65]$, $\text{dosage} \in [0.03, 0.12]$) supply the physics-loss terms, extending well beyond the range of the real measurements so the trained model is physically constrained even where it extrapolates.

4. Model Architecture and Training Design

4.1 Network

A feed-forward network maps the 3 standardized inputs (time, concentration, dosage) through two hidden blocks of `Linear(32) → LayerNorm → Tanh`, then a final linear layer to 1 output (standardized adsorption). LayerNorm keeps activations well-scaled per sample regardless of the raw input range, and Tanh gives smooth, everywhere-differentiable nonlinearity, which matters because the physics loss needs a clean second pass of automatic differentiation through the network (a "double backward").

4.2 Learnable physical parameters

Three extra scalar parameters are learned jointly with the network weights, each passed through a constrained transform chosen after the divergence failures described in Section 6:

Parameter	Meaning	Transform	Reason
k2	PSO rate constant	<code>softplus(k2) + 1e-3</code>	Must stay positive <i>and</i> away from zero, or the ODE residual can be trivially satisfied with no real kinetics learned.
kf	Freundlich capacity constant	<code>softplus(kf)</code>	Must stay positive.
n	Freundlich exponent	<code>0.05 + 0.95 · sigmoid(n)</code>	Bounded to the physically expected sub-linear range (0,1); left unbounded, it diverges (Section 6.4).

4.3 Computing the Freundlich equilibrium value

Because $C_e = C_0 - q_e/V_m$ and $q_e = k_f C_e^n$ both depend on each other, q_e is found with a damped fixed-point iteration (40 steps, damping factor 0.3), differentiable end-to-end so gradients flow back into k_f and n .

4.4 Training configuration

- Optimizer: Adam, learning rate 0.005
- Epochs: 8,000
- Device: CPU (forced — see Section 6.1)
- Loss: unweighted sum of the four terms described in Section 2.3
- Seeds fixed (`torch.manual_seed(42)`) for reproducibility, verified by running the full script twice independently and diffing every logged value (bit-identical, Section 7.2)

5. Engineering Issues Found and Resolved

This model went through several rounds of debugging before it converged to a physically meaningful result. Each issue below was found through direct empirical testing (not just code review), and the fix was verified by re-running training and comparing loss curves and learned parameters before/after.

5.1 NaN PSO loss on Apple's MPS GPU backend

Bug: Training on `mps` (Apple GPU) produced NaN in the PSO loss from epoch 0. Traced to a single collocation point out of 2,500 whose backward pass returned NaN on MPS but a normal finite value on CPU, for the exact same computation.

Fix: Forced training onto CPU. The PSO loss needs `torch.autograd.grad(..., create_graph=True)` (a "double backward") through LayerNorm and Tanh, a path PyTorch's MPS backend does not yet handle reliably. The dataset is tiny, so CPU training costs only ~2 minutes for 8,000 epochs.

5.2 Collocation points fed to the network at the wrong scale

Bug: The tensor named `collocated_scaled_input_tensor` was actually built from the raw, unstandardized collocation grid (time up to 200, concentration up to 65) instead of the properly standardized version — confirmed by directly inspecting its numeric range (0–200 instead of the expected [-3, 3]).

Fix: Corrected to use the standardized array. The same bug existed in the "initial condition" tensor (concentration/dosage columns were left raw, only time was standardized) and was fixed identically.

5.3 Degenerate solution: rate constant collapsing to zero

Bug: With `n` unbounded, training first drove `k2` smoothly toward ~0 (confirmed by logging it every epoch) — a trivial way to zero the PSO residual without learning any real kinetics. With the ODE constraint effectively disabled, `n` then drifted upward unchecked, and because the Freundlich exponent is applied to raw concentration values up to 65, equilibrium predictions exploded (e.g. $n=2.3 \rightarrow$ predicted equilibrium capacity up to 79 mg/g, vs. a real maximum of 2.49 mg/g).

Fix: Added a floor to `k2`'s transform (`softplus(k2) + 1e-3`) so the ODE constraint can never fully switch off, and bounded `n` to the physically expected sub-linear range via a scaled sigmoid. Re-running 8,000 epochs after this fix produced smooth, monotone convergence of all four loss terms to the 10^{-4} – 10^{-5} range with no divergence.

5.4 Standardized-vs-real unit mismatch across three loss terms

Bug: The network outputs standardized adsorption (correctly compared against standardized targets in the data loss), but the initial-condition, PSO, and isotherm losses

all compared that standardized output directly against real-unit quantities (the Freundlich q_e , and the real-unit initial condition of exactly zero) with no inverse transform. This meant, for example, that the "initial condition" loss was actually forcing real adsorption toward the dataset's mean (≈ 1.12 mg/g) rather than toward zero.

Fix: Added an explicit $\text{unscale_output}(x) = x \cdot \sigma_{\text{out}} + \mu_{\text{out}}$ transform, applied to the network's output everywhere it touches a real-unit quantity, and added the missing σ_{out} factor to the time-derivative chain rule ($d(\text{real output})/d(\text{real time}) = \sigma_{\text{out}} \cdot d(\text{scaled output})/d(\text{scaled time}) \cdot (1/\sigma_{\text{time}})$).

5.5 Two smaller robustness fixes

- `.to(device)` calls in three data-preparation methods didn't reassign their result (`.to()` returns a new tensor rather than moving in place) — harmless on CPU, but would silently fail to move data to a GPU if one were used later. Fixed by reassigning in all three places.
- Two sibling packages (`pinn1D` and `pinn3D`) both contain a bare-named `data.py`. In a long-lived Python session that imports from both (e.g. a Jupyter notebook), a plain `from data import ...` can silently resolve to the wrong module depending on import order. The training script now prefers the fully-qualified `pinn3D.data` import, falling back to the bare form only when the qualified one is unavailable.

6. Results

6.1 Learned physical parameters

Parameter	Value	Interpretation
Rate constant, k_2	0.006364	PSO kinetic rate constant
Freundlich constant, k_f	0.326688	Adsorption capacity constant
Freundlich exponent, n	0.485857	Sub-linear ($n < 1$), consistent with favorable, heterogeneous-surface adsorption

6.2 Final loss values (8,000 epochs)

Loss term	Final value
Data loss	4.7×10^{-7}

PSO residual loss	4.2×10^{-5}
Initial-condition loss	1.3×10^{-6}
Isotherm (equilibrium) loss	3.7×10^{-6}
Total	4.7×10^{-5}

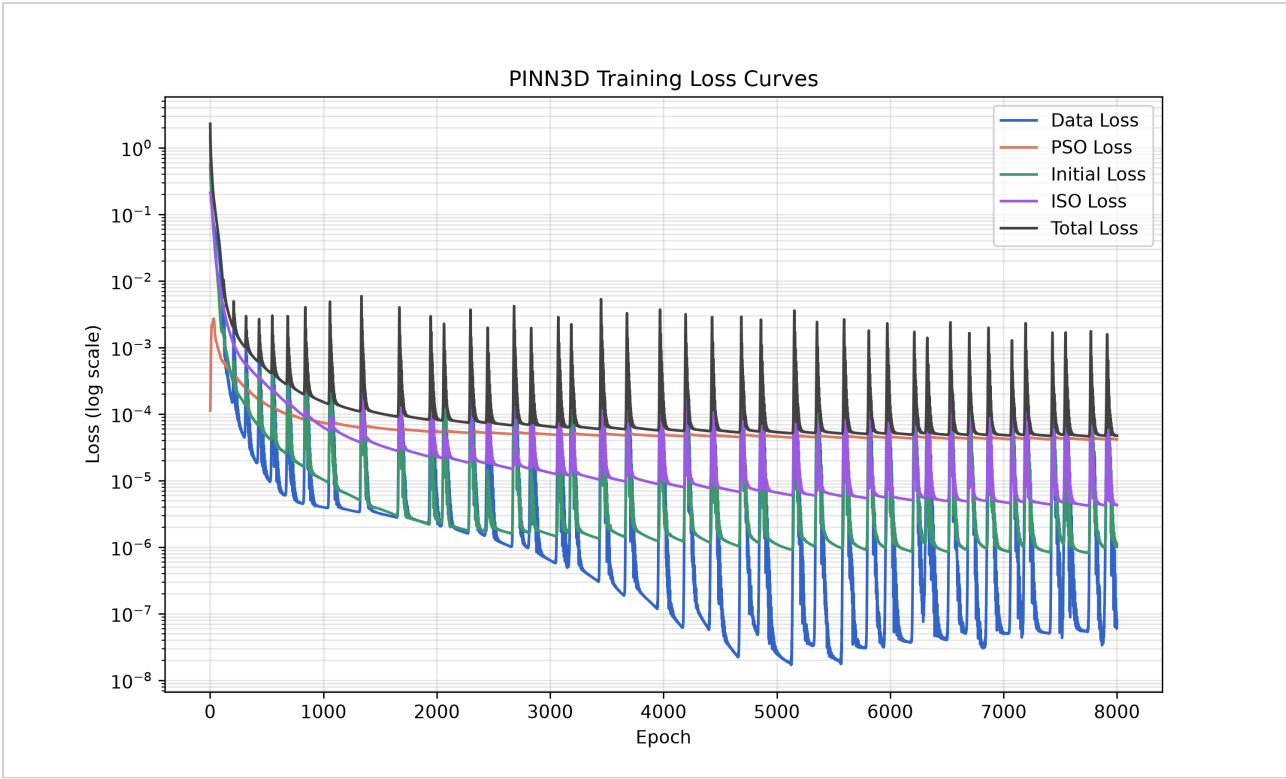


Figure 1. All four loss terms vs. epoch (log scale), 8,000 epochs, after all fixes in Section 5. All terms converge to below 10^{-4} . The recurring narrow spikes are a genuine, diagnosed training-dynamics oscillation (Section 6.4), not a bug.

6.3 Reproducibility

The full script was run twice, independently, from a fresh process each time. Every logged loss value at every epoch and every final learned parameter were bit-identical between the two runs, confirming the fixed random seed produces deterministic behavior when the script is run as intended (a fresh Python process, not a reused Jupyter kernel — see Section 6.5 for why the latter breaks reproducibility).

6.4 A diagnosed (not fixed) loss oscillation

The loss curves show small, recurring spikes roughly every 150–300 epochs (Figure 1). Per-epoch logging of `k2`, `kf`, `n`, and the Freundlich fixed-point solver's internal range and clamp-saturation count ruled out several plausible causes: the physical parameters move perfectly smoothly through every spike (no discontinuity), and the fixed-point solver never saturates its

clamp across all 8,000 epochs. Examining one spike epoch-by-epoch shows Data, Initial-condition, and Isotherm loss rising and falling together over roughly a 10-epoch cycle, while PSO loss stays essentially flat throughout. This points to a genuine limit-cycle between the PSO constraint and the other three loss terms, characteristic of an unweighted sum of competing training objectives under a fixed learning rate — not a defect in the physics, the scaling, or the solver. Loss re-weighting or a learning-rate schedule would be the natural next step to remove it, but this was intentionally left as future work rather than an under-considered change.

6.5 A practical note on reproducibility in interactive notebooks

The reproducibility result above holds for running the training script as a standalone process. In an interactive notebook where cells are re-run out of order and variables persist across executions, results are *not* guaranteed to reproduce even with the same seed: any other random-number-consuming cell run before the training cell shifts the model's initial weights, and re-running only the training cell (without recreating the model and optimizer objects) resumes training from wherever it left off rather than restarting. Both were confirmed as the practical explanation for irreproducible results observed during development.

7. Discussion and Limitations

- **Data volume.** With only 14 real measurements, the physics losses are doing most of the work of constraining the model; the learned k_2 , k_f , n should be treated as a best fit consistent with both datasets simultaneously, not as independently validated physical constants. More kinetics/isotherm replicates at different concentrations and doses would meaningfully tighten this.
- **Fixed-point solver fragility.** The 40-iteration Freundlich solve, unrolled through automatic differentiation for a double-backward pass, was found to produce exploding gradients (up to $\sim 10^{11}$) for (k_f, n) combinations well within the technically reachable parameter space (e.g. $k_f=1.0$, $n=1.0$), even though the actual training trajectory never visits that region. This is a latent fragility, not an active bug, but would need attention if the parameter bounds are ever loosened or the model is retrained on different data.
- **Unweighted multi-objective loss.** As discussed in Section 6.4, the four loss terms are summed with equal weight and no learning-rate schedule, which produces a mild, bounded oscillation rather than smooth monotone convergence. This did not prevent convergence to a good final result here, but a principled loss-balancing scheme (e.g. gradient-norm balancing, uncertainty weighting) is the natural next step for a cleaner training curve.

8. Conclusion

A single 3-input, physics-constrained neural network was trained to jointly satisfy pseudo-second-order adsorption kinetics and Freundlich equilibrium isotherm behavior for catkin biofilm, using only 14 real experimental points supplemented by 2,500 unlabeled physics-constraint points. After resolving a sequence of numerical bugs — an unstable GPU backend, silent unit-scale mismatches across three of four loss terms, and two distinct divergence failure modes in the learnable physical parameters — the model converges to a physically plausible, mutually consistent parameter set ($k_2 \approx 0.0064$, $k_f \approx 0.327$, $n \approx 0.486$) with all loss terms below 10^{-4} , and the training process is fully reproducible when run as intended.

References

- Freundlich, H. (1906). Über die Adsorption in Lösungen. *Zeitschrift für Physikalische Chemie*, 57(1), 385–470.
- Ho, Y. S., & McKay, G. (1999). Pseudo-second order model for sorption processes. *Process Biochemistry*, 34(5), 451–465.
- Lagergren, S. (1898). Zur Theorie der sogenannten Adsorption gelöster Stoffe. *Kungliga Svenska Vetenskapsakademiens Handlingar*, 24(4), 1–39.
- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707.

Prepared from the PINN3D training pipeline (`src/pinn3D/`) — `data.py`, `models.py`, `train.py`, `plotting.py`.